



Stamin_Backend_ML/

- |
- |—— data/ # all the training text/files that we want the model to learn
- |—— models/ # trained model files are stored (eg- saving progress in a video game)
- |—— src/
 - |—— dataset.py # loads & preprocesses your text data
 - |—— tokenizer.py # converts plain texts data into numerical format
 - |—— model.py # your neural network architecture
 - |—— train.py # training loop
 - |—— infer.py # code to generate output from model
- |
- |—— app.py # Flask/FastAPI for ML APIs
- |—— requirements.txt
- |—— README.md



It stores plain training text files that your model will learn from.

Why not a database?

Because during training, the model needs to read large amounts of text very fast, and reading from files is much faster, simpler, and standard than reading from a database.

Databases are for apps and websites.

Plain text files are for model training.

Real-world example

Training data in data/ is like having a recipe book directly in your kitchen.

You don't go to a restaurant database to read recipes while cooking — you keep simple text recipes right next to you for fast access.



models/ is the folder where your trained model gets saved after training.

Short explanation -

It stores the model's weights/checkpoints, so you can load the model later without retraining.

Real-world example

Like a game save file — you don't want to start the game from the beginning every time, so you save your progress.



`dataset.py` is the file that prepares your training data so the model can understand it.

In short -

- Reads your text files from the `data/` folder
- Cleans the text (removes extra spaces, symbols if needed)
- Converts text into tokens/IDs using the tokenizer
- Packs them into batches that the model can learn from

Real-world example

Like a kitchen helper who washes, cuts, and arranges ingredients before the chef (model) starts cooking



A tokenizer is needed because a model cannot understand raw text like :-
→ "Suggest a diet for muscle gain" , It only understands numbers.

Therefore -

The tokenizer converts text → numbers (tokens) so the model can learn from it.

Real-world example

It's like translating English words into IDs so a computer can process them.



`model.py` is the file where you build the neural network itself.

Short explanation -

It defines how your model looks — layers, attention blocks, embeddings, etc.
This is the brain of your AI.

Real-world example

Like designing the blueprint of a robot — its sensors, circuits, and how it thinks.
Everything about how the model works internally is written here.



`train.py` is the file that teaches your model using the data.

Short explanation -

It runs the training loop:

- send tokenized data into the model
- compute loss
- update weights
- repeat again and again

This is where the actual learning happens.

Real-world example

Like a gym trainer who makes you exercise every day — reps, sets, corrections — so you become stronger over time.



`infer.py` is the file that lets your trained model generate answers.

Short explanation

It loads the saved model and uses it to predict text, like diet plans, workouts, or replies.

Real-world example

If `train.py` is the teacher,
then `infer.py` is the student giving answers in an exam after learning.



`app.py` is the file that makes your model available to the outside world through APIs.

Short explanation

It creates API routes like `/diet`, `/workout`, `/ask-ai` so your mobile app can send a question → model processes it → returns an answer.

Real-world example

Like a reception desk in a gym — people come with questions, the receptionist forwards them to the trainer (your model), and returns the response.



requirements.txt lists all the Python libraries your project needs.

Short explanation

When someone installs your project, this file tells Python which packages to install (like torch, numpy, flask, etc.).

Real-world example

It's like a shopping list for your project — everything needed to make it run properly.